

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192525304
Name	D.Raghava

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I D.Raghava – 192525304 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D.Raghava

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Q1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Answer

The likely design error is **poor input partitioning and unbalanced workload distribution**. If input splits are highly uneven, some Mapper tasks receive much more data than others. These Mappers take longer to finish, while other tasks complete early. As a result, reducers may remain idle waiting for sufficient intermediate data.

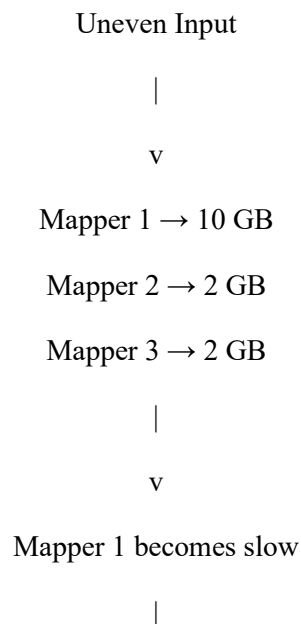
Causes

1. **Uneven input split sizes** cause workload imbalance.
2. Poor partitioning may send too much data to a particular Mapper.
3. **Data skew** can cause some keys to produce much larger intermediate data.
4. An inappropriate number of reducers can leave reducers underutilized.
5. Excessive intermediate data can increase shuffle and network overhead.

Corrective Actions

- Configure appropriate HDFS block and input-split sizes.
- Increase the number of Mapper tasks when sufficient cluster resources are available.
- Use an appropriate **partitioner** to distribute keys more evenly.
- Use a **Combiner** where applicable to reduce intermediate data.
- Select a suitable number of Reducers based on the input size and cluster capacity.
- Monitor task execution to identify stragglers and overloaded partitions.

Example



v

Reducers wait / remain idle

After correction:

Balanced Input

|

v

Mapper 1 → 4.5 GB

Mapper 2 → 4.5 GB

Mapper 3 → 5 GB

|

v

Balanced Shuffle

|

v

Reducers work efficiently

Conclusion

The main error is **unbalanced input and poor task distribution**. Proper input splitting, balanced partitioning, suitable reducer configuration, and combiners can improve parallelism and prevent reducers from remaining idle.

Q2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

Answer

Frequent data unavailability after the failure of only one node indicates a problem with **HDFS replication, block placement, or NameNode availability**.

Probable Errors

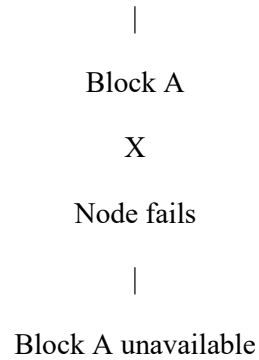
1. Insufficient Replication

If the replication factor is set too low, a single DataNode failure can make blocks unavailable.

For example:

Replication Factor = 1

DataNode 1



A higher replication factor provides additional copies.

Replication Factor = 3

DN1 → Block A

DN2 → Block A

DN3 → Block A

If DN1 fails, DN2 and DN3 can still provide the block.

2. Poor Replica Placement

If replicas are placed on the same physical failure domain, such as the same rack, a rack-level failure can affect multiple copies.

Rack-aware placement should be used.

3. NameNode Single Point of Failure

If the deployment has only one NameNode, its failure can make the HDFS namespace inaccessible even when DataNodes are healthy.

An **Active–Standby NameNode** design should be used for high availability.

Corrective Actions

- Configure an appropriate HDFS replication factor.
- Use rack-aware block placement.
- Monitor under-replicated blocks.
- Enable automatic re-replication after DataNode failure.
- Use Active and Standby NameNodes for NameNode high availability.
- Regularly monitor DataNode and NameNode health.

Conclusion

The likely design error is **insufficient replication and/or a single-NameNode architecture**. Proper block replication, rack awareness, automatic re-replication, and NameNode high availability can prevent a single node failure from causing frequent data unavailability.

Q3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Answer

Duplicate records indicate that the ingestion pipeline does not correctly maintain **record identity, processing state, or idempotency**.

Likely Causes

1. Repeated Data Ingestion

The same source records may be processed multiple times because the pipeline does not track which records have already been successfully ingested.

2. Missing Unique Identifier

If records do not have a unique key, the analytics store may not be able to distinguish new records from previously processed records.

3. NiFi Flow Configuration

A NiFi flow may accidentally send the same data through multiple paths or retry successful transfers incorrectly.

4. ETL Retry Problems

If an ETL process fails after writing data but before recording successful completion, it may retry the same records.

5. Full Extraction Instead of Incremental Extraction

Repeatedly extracting the entire source dataset can cause previously loaded records to be inserted again.

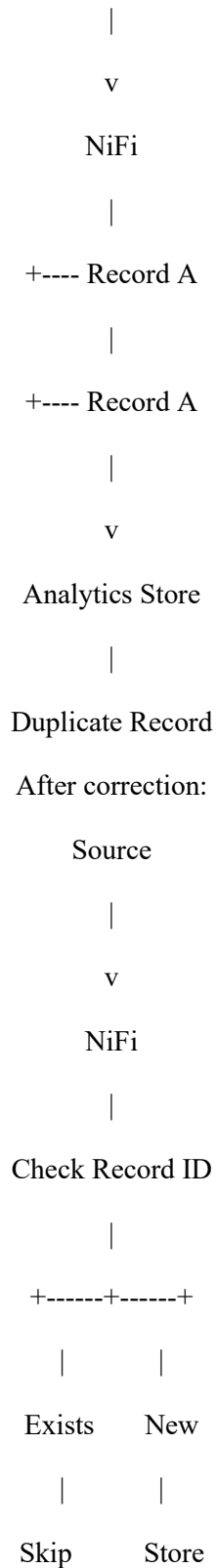
Corrective Actions

- Assign a **unique record ID** to each record.
- Use incremental extraction based on timestamps, sequence numbers, or change tracking.
- Maintain ingestion state/checkpoints.
- Configure NiFi relationships and retries carefully.
- Perform deduplication before loading data.
- Use an idempotent loading mechanism in the analytics store.

Example

Before correction:

Source



Conclusion

The primary error is the absence of **duplicate detection and reliable ingestion-state management**. Unique identifiers, incremental ingestion, checkpoints, controlled retries, and idempotent loading can prevent duplicate records.

Q4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

Answer

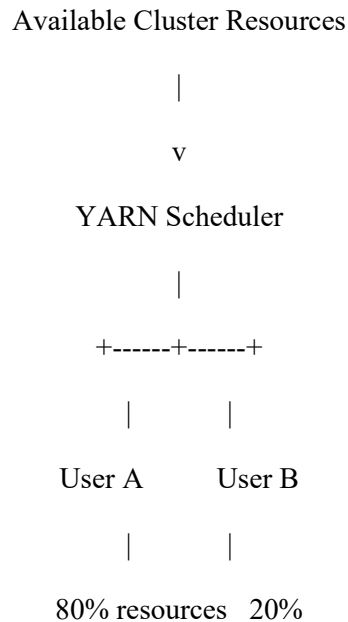
Resource starvation occurs when some users or applications consume most of the available YARN resources, leaving insufficient CPU and memory for other jobs.

The likely error is **poor scheduler configuration or lack of resource allocation policies.**

Causes

1. No proper queue configuration.
2. One user or application consumes excessive resources.
3. No minimum resource guarantees for different workloads.
4. Poorly configured priorities.
5. Lack of fair sharing between users.
6. Insufficient monitoring of CPU and memory usage.

Example



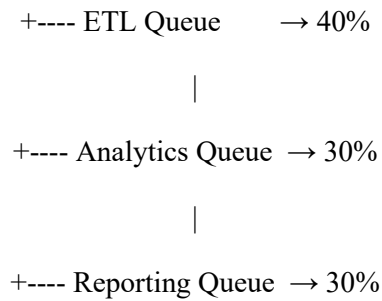
If User A continuously submits large jobs, User B may experience starvation.

Corrective Actions

A **Fair Scheduler or Capacity Scheduler-style policy** should be used with appropriate queues and resource limits.

Example:

YARN

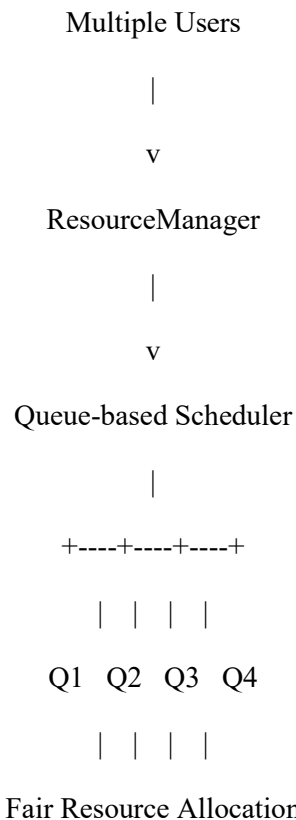


The exact percentages should be configured according to the organization's workload.

Additional controls include:

- Set minimum resources for important queues.
- Set maximum resource limits.
- Use job priorities carefully.
- Apply fair sharing between users.
- Prevent a single application from consuming the entire cluster.
- Monitor CPU, memory, and container utilization.
- Configure preemption where appropriate.

Improved Scheduling



Conclusion

The scheduling error is **uncontrolled resource allocation and lack of fairness or queue management**. A properly configured YARN scheduler with queues, resource limits, priorities, and fair sharing can prevent resource starvation and provide predictable performance for multiple users.

Q5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Answer

Restoring outdated data indicates that the backup system is not correctly maintaining or selecting the **latest valid version of the data**.

Probable Design Flaws

1. Stale Backup Copies

The backup may be created infrequently, so the latest changes are not included.

Day 1 → Backup A

Day 2 → Data Updated

Day 3 → Data Updated

Day 4 → Failure

|

v

Restore Backup A

|

v

Old Data

2. Incorrect Replication Assumption

Replication is not the same as a complete backup strategy. Replicas may contain the same logical state and may not provide historical recovery from corruption or accidental deletion.

3. No Versioning

If backups overwrite older copies without retaining appropriate recovery points, the system may not be able to restore the correct version.

4. Failed Backup Validation

A backup may appear successful but contain incomplete or inconsistent data.

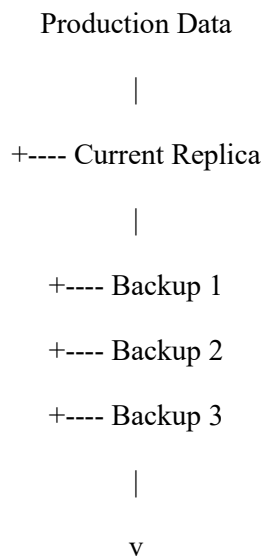
Corrective Actions

- Schedule backups at appropriate intervals.

- Maintain multiple recovery points.
- Use backup versioning.
- Verify backup completion and integrity.
- Monitor backup timestamps and metadata.
- Test restoration periodically.
- Maintain appropriate replication for availability while using independent backups for recovery.
- Clearly define **Recovery Point Objective (RPO)** and **Recovery Time Objective (RTO)**.

Example

Improved strategy:



Multiple Recovery Points

If the latest data is corrupted, the organization can select an appropriate valid recovery point instead of restoring an unnecessarily old copy.

Replication vs Backup

Replication	Backup
Primarily supports availability	Primarily supports recovery
Maintains copies of current data	Maintains recovery versions
Helps with node failures	Helps with corruption/deletion
Usually near-real-time	Usually scheduled/versioned

Conclusion

The probable flaw is **stale or improperly managed backups and confusion between replication and backup**. The solution is to use timely, versioned, validated backups with multiple recovery points while maintaining replication for high availability.